

Device Passthrough Investigation & Implementation on SMMU-Supported pKVM

ShangJhe Li MingLong Tsai YuSiang Deng

Department of Computer Science and Information Engineering
National Taiwan University, Taipei, Taiwan

Abstract

While the protected Kernel-based Virtual Machine (pKVM) framework provides strong CPU-level memory isolation through Stage-2 page tables, its protection against Direct Memory Access (DMA) attacks from hardware peripherals remains incomplete without a trusted Input/Output Memory Management Unit (IOMMU).

Recent public patch series have proposed a hypervisor-owned Arm System Memory Management Unit version 3 (SMMUv3) driver for pKVM. This driver manages SMMU Stage-2 page tables at EL2 and enables the hypervisor to enforce DMA access control independently of a potentially compromised host kernel. Building on this foundation, this project investigates whether the updated pKVM SMMUv3 integration is sufficient to support secure device passthrough. As a baseline, we target network interface card (NIC) passthrough, enabling a guest VM to directly control a physical NIC with near-native performance while maintaining strict cross-VM memory isolation enforced by the SMMU. We also analyze remaining gaps in the prototype, such as support for translated domains and per-device isolation granularity, and outline the extensions required for broader device passthrough support in virtualized embedded platforms.

Keywords: pKVM · SMMU · Device Passthrough

1 Introduction

Confidential computing protects a workload’s data and execution from the privileged software around it, including a compromised operating system or hypervisor. On Arm, the pKVM provides this on commodity Armv8.0 and later cores without specialized confidential-computing hardware: a small component runs at EL2, independent of the much larger host kernel at EL1, and the second stage of the Arm MMU deprives the host after boot, so EL2 can revoke a protected guest’s

pages from the host and keep guest memory confidential even if the host is later subverted [1], [2], [3]. That protection only covers accesses that flow through a CPU’s MMU. A DMA-capable peripheral issues its own memory transactions, and without an IOMMU under trusted control it can read or write arbitrary physical memory [4], which lets a compromised host aim a device at guest or hypervisor memory and bypass Stage-2 entirely [3]. Recent patch-series work closes this hole by moving control of the Arm System Memory Management Unit version 3 (SMMUv3) into the hypervisor: a reduced EL2 driver populates the SMMU’s tables and command queue while the host retains probing and some initialization [5], [6], and EL2 shadows the host’s stream table and traps the command queue so every device stream goes through a Stage-2 translation it controls. In the baseline that translation is a single host identity map, so the host can no longer reach protected memory.

That base isolates the host’s own devices but says nothing about handing a device to a guest. Device passthrough, where a guest VM drives a physical device directly instead of through a paravirtualized interface, matters most for performance-sensitive devices. Under pKVM it requires the device’s DMA to stay confined to the assigned guest’s Stage-2 address space no matter what the untrusted host does. This paper asks whether the updated pKVM SMMUv3 integration is enough for secure passthrough, what must be added to achieve it, and what stays out of reach on commodity hardware. Our contributions are:

1. We extend the hypervisor-owned pKVM SMMUv3 base, which confines host devices to a Stage-2 identity map, into a passthrough mechanism: EL2 redirects an assigned stream to a guest’s Stage-2, computing the routing decision and the stream-table contents entirely from trusted hypervisor state.

2. We add an attach/detach hypercall interface and wire it into the standard Linux passthrough stack (VFIO and IOMMUFD), so binding a device to `vfio-pci` and handing it to a protected VM installs the EL2 assignment with no guest-visible change.
3. We identify and fix the correctness and safety problems passthrough introduces: reconciling the guest and SMMU Stage-2 configurations (VTCR and IPA-size clamping), validating host-supplied StreamIDs, restricting host TLB invalidations to the host VMID, managing VMID invalidation across (re)assignment, and detaching devices before a VM's memory is reclaimed.
4. We port the mechanism onto a Linux 6.18-based pKVM SMMUv3 branch and validate it functionally under QEMU with an emulated SMMUv3 and a VFIO endpoint; a quantitative performance study is left as future work.

2 Background

2.1 Protected KVM

pKVM (protected KVM) is the Android hypervisor that separates the host Linux kernel from the code which actually enforces VM isolation. pKVM runs at EL2, the highest privilege level on ARM, and everything else, including the full host kernel, runs above it as an ordinary guest. Thus, even if the host kernel is compromised, it can't read or modify a protected VM's pages. [1]

2.2 The Arm SMMUv3

The Arm SMMUv3 is the IOMMU defined by the third version of Arm's System MMU architecture. [7] It sits between DMA-capable devices and memory, translating virtual addresses into physical addresses and enforcing which memory each device is allowed to touch. Unlike SMMUv2, v3 keeps its configuration in memory: a Stream Table indexed by StreamID, where each Stream Table Entry(STE) can point at Context Descriptors that hold the per-context stage-1 translation state. It supports the same two translation stages as the MMU in Arm CPU: stage 1 translates virtual to intermediate-physical addresses (typically owned by a guest or OS) and stage 2 translates intermediate-physical to physical (owned by the hypervisor). SMMUv3 enables the hypervisor to give a VM direct device access while keeping that VM's DMA confined to its own memory. Software drives the unit through in-memory queues rather than registers: a Command Queue for configuration changes

and TLB invalidations, an Event Queue where the SMMU reports translation faults, and an optional Page Request Interface queue. A StreamID identifies the device or stream, and a SubstreamID (the equivalent of a PCIe PASID) selects a context within that device, which together with ATS and PRI support shared virtual addressing between a device and a process.

2.3 The hypervisor-owned SMMUv3 driver

Preventing host DMA from bypassing Stage-2 requires the SMMU to be owned by a trusted entity, which in pKVM is the EL2 hypervisor. Relocating the full SMMUv3 driver into the minimal EL2 component is impractical, since it is large and depends on infrastructure spread across the kernel. The solution is a split driver: a reduced EL2 component that only populates SMMU tables and issues commands, and a host component that handles probing and setup [5], [6]. Rather than expose the SMMU registers to the host, the hypervisor shadows the host's stream table. The host writes STEs in its own memory, and a trap-and-emulate path filters the SMMU command queue and reflects those writes into a hypervisor-private shadow table, rewriting each stream to pass through a Stage-2 that EL2 controls [6]. In the baseline patch series, that Stage-2 is a single host identity map tagged with VMID 0, and the host commands that would weaken isolation are filtered out. Every host device is therefore confined to the identity map: a stream points at the host identity map, never at a guest, and cannot reach protected memory. Our work builds on this machinery.

2.4 VFIO and IOMMUFD

VFIO is the Linux framework that exposes a device safely to userspace or a guest, and IOMMUFD is its newer IOMMU control plane [8]. To do a device passthrough on a VM, the hypervisor binds the device to the `vfio-pci` driver, which detaches it from its host driver and attaches it to an IOMMU domain the hypervisor controls; the device's DMA is then constrained by that domain's page tables. We integrate the EL2 assignment into this same path, so existing passthrough tooling drives it.

3 Threat Model and Goals

We adopt the pKVM trust model and extend it to an assigned device. The hypervisor, the SMMU hardware, and the reduced EL2 SMMU driver are trusted. The EL1 host Linux kernel and the VMM are untrusted: they still schedule virtual CPUs, manage resources,

and mediate passthrough through the VMM’s device assignment, but they must not read or write guest or hypervisor memory. We assume a powerful host adversary that can program the host-visible stream table arbitrarily, drive any host-owned DMA device, and issue SMMU commands, only subject to the hypervisor’s filtering. We consider one physical DMA-capable device, a network interface card in the baseline, assigned to a protected guest. The device and the interconnect are trusted: the device only issues transactions under its own StreamID and does not spoof another device’s SID, and we assume no cryptographic protection of the link.

Under these assumptions, the design targets three security goals. **G1.** An assigned device’s DMA stays within the assigned guest’s Stage-2 address space and cannot reach another guest’s or the hypervisor’s memory. **G2.** The host cannot use the assignment mechanism or the assigned device to reach guest or hypervisor memory; in particular, it cannot redirect a stream to a Stage-2 the corresponding guest does not own, nor disturb a guest’s SMMU translations. **G3.** When a guest is destroyed, its devices stop translating through its Stage-2 before that memory is reclaimed.

A malicious or buggy device, an endpoint that spoofs StreamIDs, or an adversary that eavesdrops on or tampers with the PCIe fabric are out of scope, since we provide no link encryption. We do not address side channels, physical attacks, or the confidentiality of the device’s own register file beyond its DMA behavior. Finally, availability is not a goal: an untrusted host can always decline to schedule a guest, which is consistent with the base pKVM model.

4 Design

4.1 Overview

The base hypervisor-owned SMMU always routes a stream to the host identity-map Stage-2. Our design instead lets EL2 route an assigned stream to the target guest’s Stage-2. Because the hypervisor already maintains the guest’s Stage-2 (the IPA-to-PA mapping that defines the guest’s view of physical memory), and because the SMMU can walk that same Stage-2, confining a device to the guest’s Stage-2 also confines its DMA to the guest’s IPA space. The routing decision and the contents of the resulting STE are computed entirely at EL2 from trusted hypervisor state, namely the guest’s VMID, Stage-2 base and translation control.

The untrusted host can request an assignment and can keep managing its own view of the stream table, but it cannot determine where a stream actually translates. Fig. 1 shows the arrangement.

4.2 Per-stream assignment state

EL2 keeps a compact assignment table that maps a StreamID to the identity of its owning guest and to a snapshot of that guest’s Stage-2 parameters. The snapshot is taken at assignment time from the guest’s Stage-2 MMU state, so that the latency-sensitive stream-table shadowing path can rebuild a stream’s STE without re-acquiring the global per-VM table lock. Table 1 lists the fields of a STE. Recording the owning VM handle alongside the Stage-2 parameters lets the hypervisor find and detach all of a VM’s streams when the VM is destroyed.

4.3 Attach and detach interface

Two hypercalls reach EL2 over SMCCC and are dispatched through the existing per-driver IOMMU operations table, so the mechanism is independent of any particular IOMMU. The SMMUv3 backend implements the operations. Attach looks up the SMMU and the target VM, validates the SID, snapshots the Stage-2 parameters, installs or updates the assignment, and reshadows the live STE so the stream begins translating through the guest Stage-2. Detach drops the assignment and reshadows the STE back to the host identity map.

Hypercall and dispatch interface. The host requests assignment over SMCCC, and EL2 dispatches through the IOMMU operations vtable:

```
/* EL1 -> EL2 over SMCCC */
int pkvm_iommu_attach_dev
    (phys_addr_t smmu_base, u32 sid, pkvm_handle_t handle);
int pkvm_iommu_detach_dev(phys_addr_t smmu_base, u32 sid);

/* dispatched through per-driver IOMMU operations table */
struct kvm_iommu_ops {
    int (*attach_dev)
        (phys_addr_t smmu_base, u32 sid, pkvm_handle_t handle);
    int (*detach_dev)(phys_addr_t smmu_base, u32 sid);
    void (*detach_dev_all)(pkvm_handle_t handle);
};
```

4.4 Integration with stream-table shadowing

The assignment is consulted from the same routine that shadows an STE. A stream is (re)shadowed in two situations: immediately after an attach, and whenever the host reprograms the stream’s host-visible STE and the command-queue filter reflects that change into the shadow.

Field	Meaning
sid	StreamID of the assigned device
handle	pKVM handle of the owning protected VM
vmid	VMID reused from the guest Stage-2; tags the device’s SMMU TLB entries
pgd_phys	Physical base of the guest Stage-2 page table
vtcr	Guest VTCR_EL2 bits, used to derive the STE Stage-2 control

Table 1: fields in a Stream Table Entry (struct `smmu_dev_assign`).

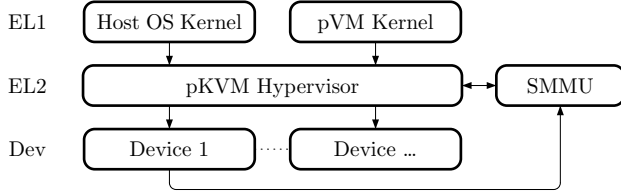


Fig. 1: System overview

In both cases the EL2 driver checks the assignment table. If the SID is assigned, it builds an STE that performs Stage-2 translation using the guest’s snapshotted parameters; otherwise it falls back to the host identity map. This behavior enforces goal G2: an assigned stream is pinned to the guest Stage-2 regardless of what the host writes into its own view of the stream table. The host can continue to manage its stream table, but it cannot, by writing an STE, cause an assigned device to translate anywhere other than where EL2 decided.

4.5 Reconciling guest and SMMU Stage-2 configuration

A guest’s Stage-2 is sized for its IPA, which comes from the CPU’s capabilities, and the SMMU may have a smaller input or output address size than the CPU. The SMMU’s VTCR subfields share the CPU’s VTCR_EL2 bit positions, so the hypervisor derives the STE Stage-2 control from the guest’s VTCR_EL2 directly. It must also clamp the effective address size to what the SMMU can walk:

$$\text{ipaeff} = \min(\text{iassmmu}, \text{oas}_{\text{smmu}}) \quad (1)$$

If the guest’s IPA exceeds what the SMMU supports, the hypervisor narrows the Stage-2 address-size field and adjusts the output-size encoding to match, keeping the SMMU’s walk consistent with the page tables built for the guest. Otherwise the STE would encode something the SMMU cannot honor.

4.6 VMID and TLB management

The SMMU caches Stage-2 translations tagged by VMID. We reuse the guest’s CPU VMID for its assigned streams, so the SMMU and CPU Stage-2 caches share one VMID namespace for the guest. Stale translations break correctness and isolation, so the hypervisor invalidates VMIDs when they go stale: moving a live stream to a new VMID flushes both old and new, and detach flushes the departing VMID before the host identity map takes over. The reverse must also hold, so the host cannot flush or poison a guest’s translations: the command-queue filter allows host-issued Stage-2 and non-hypervisor TLB invalidations only for VMID 0, the host identity map, and rejects any targeting another VMID. This enforces G2.

4.7 Lifecycle safety on teardown

Destroying a protected VM reclaims its pages, including the Stage-2 page-table pages, for reuse. First the hypervisor must ensure no assigned device still walks the guest’s Stage-2, or the device could DMA into memory already handed back. So teardown detaches every stream owned by the VM’s handle, reshadows each back to the host identity map, and flushes its VMID before any page is reclaimed. Removing the per-VM table entry first blocks concurrent attaches to that handle, closing a teardown race and satisfying G3.

4.8 Host-stack integration

The hypercalls hook into the host’s VFIO and IOMMUFD attach path. When the VMM attaches a vfio-pci device to a protected VM, the ordinary IOMMUFD attach still programs the host-visible domain, keeping the host driver’s bookkeeping intact, while a host-side bridge records the assignment at EL2 for each of the device’s streams. Detach and unbind reverse this. Passthrough runs through the standard Linux interface, and the guest sees an ordinary assigned device.

5 Implementation

We implemented the design as a patch series on a pKVM SMMUv3 development branch. The work spans the hypervisor, the host SMMU driver, the generic IOMMU layer, and the host’s device-passthrough path.

5.1 Assigning a device to a guest

The host can ask the hypervisor to hand a device’s DMA streams to a protected VM, or to take them back. These requests travel from the host down to the hypervisor through a small hypercall interface, but the hypervisor remains the only entity that touches the real stream table. It keeps a private record of which streams belong to which VM and consults that record whenever it programs the hardware.

5.2 Confining device DMA

When a stream is assigned, the hypervisor programs it to translate through the guest’s own Stage-2 page tables under the guest’s VMID, so the device can reach only the memory that guest already owns. Unassigned streams keep the host’s existing identity mapping. Because the host is untrusted, everything it supplies is checked first: stream identifiers are bounds-checked against the device’s table, and the guest’s address size is clamped to what the hardware supports, so the host cannot trick the hypervisor into building an invalid or over-broad entry. The hypervisor also owns TLB invalidation for guest VMIDs, and the host may only invalidate its own mapping. A device that is already live can be moved to another guest in place, without tearing down and rebuilding its mapping.

5.3 Reusing the standard passthrough path

Devices are assigned the usual way, through the kernel’s VFIO/IOMMUFD interface. We hook that path so that, when protected mode is active, attaching a device to a VM quietly drives the assignment mechanism above. When pKVM is not in use, the normal behavior is unchanged.

6 Evaluation

We validate the design with two demonstrations that run the full device-to-guest path under QEMU 11.0.1 in pure TCG, with L0 emulating EL2, a GICv3, and an SMMUv3, and the L1 pKVM host in protected mode (`kvm-arm.mode=protected`). The stack is software on software, so the timings are slow by construction and we read nothing into them.

In the first demonstration (Proof A), a self-contained init brings up a VM and passes an emulated `iommu-testdev` endpoint into it via `vfio-pci`, using the VFIO type1 group attach; the `iommufd-cdev` attach returned `EINVAL` here, so the type1 path drives the EL2 assignment. The device DMA a known value into the VM’s page and the VM’s vCPU reads it back correctly, exercising the hypercall dispatch, the EL2 assignment, the STE reshard, and the VFIO integration without faulting.

In the second demonstration (Path B), a full Ubuntu L2 booted by L1’s QEMU receives an L0-emulated `e1000e` NIC through `vfio-pci`. The device enumerates (`8086:10d3`), the driver binds, the link comes up at 1 Gbps, and `ping` returns four of four replies with no loss, so the NIC moves real traffic through the same L0 SMMU, programmed via L1’s type1 container and pKVM’s SMMU bridge. This L2 runs with `-enable-kvm` and is not a pVM.

The remaining quantitative study needs an additional assigned NIC. There we plan to measure throughput and latency against native and a paravirtualized baseline, the attach and detach cost, and any per-translation overhead. We leave this as a future work.

7 Conclusion

We investigated whether the proposed hypervisor-owned pKVM SMMUv3 integration is sufficient for secure device passthrough, and found that with a focused EL2 device-assignment mechanism it is. An assigned stream can be pinned to a guest’s Stage-2 entirely from the trusted hypervisor, integrated with the standard VFIO and IOMMUFD path, and kept safe across the VM lifecycle. We identified and resolved the correctness problems this raises, namely reconciling the guest and SMMU Stage-2 configurations, validating host-supplied StreamIDs, managing VMID invalidation, restricting host TLB invalidations to the host VMID, and ordering teardown before page reclamation, and we delimited what remains for production passthrough on commodity Arm, including device attestation and link encryption, finer isolation granularity, register and interrupt isolation, and nested domains. We hope this clarifies the distinction between host DMA isolation and guest device passthrough on Arm pKVM, and offers a concrete, commodity-hardware path toward the latter.

Acknowledgments. We thank the instructor Shih-Wei Li and teaching assistants of the Virtual Machines course in Spring 2026 at National Taiwan University, and the pKVM and SMMUv3 developers whose work this project builds upon.

References

- [1] The Linux Kernel Community, “Protected KVM (pKVM).” Accessed: Jun. 17, 2026. [Online]. Available: <https://docs.kernel.org/virt/kvm/arm/pkvm.html>
- [2] Android Open Source Project, “Android Virtualization Framework Architecture.” Accessed: Jun. 17, 2026. [Online]. Available: <https://source.android.com/docs/core/virtualization/architecture>
- [3] Android Open Source Project, “Virtualization Security Model.” Accessed: Jun. 17, 2026. [Online]. Available: <https://source.android.com/docs/core/virtualization/security>
- [4] A. T. Markettos *et al.*, “Thunderclap: Exploring Vulnerabilities in Operating System IOMMU Protection via DMA from Untrustworthy Peripherals,” in *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*, 2019. doi: 10.14722/ndss.2019.23194.
- [5] J.-P. Brucker, “KVM: Arm SMMUv3 Driver for pKVM.” Accessed: Jun. 17, 2026. [Online]. Available: <https://lwn.net/Articles/921869/>
- [6] M. Saleh, “KVM: arm64: SMMUv3 Driver for pKVM (Trap and Emulate).” Accessed: Jun. 17, 2026. [Online]. Available: <https://patchew.org/linux/20250819215156.2494305-1-smostafa%40google.com/>
- [7] Arm Ltd., “Arm System Memory Management Unit Architecture Specification, SMMU Architecture Version 3.” Accessed: Jun. 17, 2026. [Online]. Available: <https://developer.arm.com/documentation/ihl0070/latest/>
- [8] The Linux Kernel Community, “VFIO – Virtual Function I/O and IOMMUFD.” Accessed: Jun. 17, 2026. [Online]. Available: <https://docs.kernel.org/driver-api/vfio.html>